RadeonPilot - Project Specification

RadeonPilot — Project Specification

Track 2: Development & Local Deployment of Private AI Agents AMD AI DevMaster Radeon GPU Hackathon 2026

---

1. Application scenarios

RadeonPilot is a private, local-first **inference-operations copilot** for AMD Radeon GPUs. It turns a single Radeon workstation or cloud instance into a self-inspecting serving environment: a team of agents diagnoses the ROCm stack, plans and verifies the local model endpoint, benchmarks it under load, finds the bottleneck, and proposes an evidence-cited optimization plan.

Who it serves and why it must be private:

- **ML engineers / SREs** operating local inference need fast, trustworthy answers about their exact GPU, driver, and endpoint — not generic advice.
- **Privacy**: environment telemetry, configuration, and benchmark results can be sensitive. RadeonPilot keeps the entire agent loop on the machine; nothing leaves localhost, matching the Track 2 private-agent mandate.
- **Alignment with AMD ROCm.AI**: AMD's own roadmap brings agent workflows into ROCm (environment checks, deployment, diagnostics, optimization). RadeonPilot is a concrete Track 2 realization of that direction, running on consumer/workstation Radeon hardware rather than data-centre Instinct.

Concrete user journeys:

1. "Diagnose my Radeon inference stack" -> Environment Doctor reports GPU, ROCm version, VRAM, and endpoint process health with attestation.
2. "Deploy Qwen3-8B within my 48 GB budget" -> Deployment Planner proposes engine, context length, memory utilization, and launch flags grounded in the evidence.
3. "Why is time-to-first-token high?" -> Benchmark Interpreter reads measured records; Optimization Analyst proposes prefix caching / chunked prefill / bounded context, each cited to local knowledge.

4. Follow-up: “What if concurrency doubles?” -> the persistent session replans from memory instead of re-deriving the environment.

2. Agent architecture diagram

```
Gradio Ops Console (127.0.0.1)
   |
Python controller (src/radeonpilot)
   |-- deterministic tools   (rocm-smi / amd-smi / endpoint probes / permission gates)
   |-- local BM25 RAG        (data/knowledge/*.md)
   |-- SQLite memory + audit (runtime/radeonpilot.sqlite3)
   v
Localhost Pi HTTP bridge (pi-bridge, Node 22)
   |-- EnvDoctor --|
   |-- DeployPlanner --|      (four specialists in parallel)
   |-- BenchmarkRunner --|
   |-- OptimAnalyst --|
   v
OpsCommander (synthesis + gated action plan)
   v
OpenAI-compatible local endpoint -> AMD Radeon GPU + ROCm 7.2.1
```

Component roles:

- **Gradio Ops Console**: chat, diagnostics, knowledge search, and an Audit & Runtime panel showing the live attestation badge.
- **Python controller** (`src/radeonpilot`): gathers deterministic facts, retrieves evidence, persists memory, and prepares the bounded evidence pack for the agents.
- **Pi bridge** (`pi-bridge`): hosts five isolated Pi Agent Harness 0.83.0 sessions. Four specialists run in parallel; OpsCommander synthesizes them into a gated, ordered action plan. All Pi file/shell/edit/browser/network tools are disabled.
- **Local endpoint**: vLLM (BF16) or llama.cpp (GGUF) served on 127.0.0.1 over ROCm.

3. Introduction to core capabilities

Track 2 capability	RadeonPilot implementation
Local knowledge retrieval	BM25-style retriever over a local ROCm/vLLM tuning corpus; evidence carries document IDs, revisions, and content hashes
Tool invocation	GPU environment probe, endpoint health probe, real latency/throughput benchmark,

Track 2 capability	RadeonPilot implementation
	permission gate; tools compute facts the model cannot invent
Multi-step planning	Four specialists + OpsCommander produce an ordered plan; hazardous steps are blocked pending human approval
Local multi-turn memory	SQLite session turns, environment/action snapshots, and audit events with explicit lineage
Permission & privacy	Least-privilege role gate, localhost-only bridge and endpoint, disabled Pi tools, redacted audit
Radeon core inference	Every agent generation is served by the local ROCm endpoint; attestation binds the endpoint PID to `amd-smi process` on a Radeon device
Speed optimization	Baseline vs optimized vs quantized measured comparison; see section 5
Honest modes	`demo` (labelled synthetic) and `pi` (fail-closed, never substitutes demo)

4. Model introduction & local deployment plan

Model: Qwen3-8B, an 8-billion-parameter open-weight instruct model. It is small enough to serve comfortably on a single 48 GB Radeon in BF16 while leaving headroom for the KV cache and concurrent agent sessions, and it has a clean GGUF path for the optional quantized bonus profile.

Deployment plan on the competition instance (gfx1100, ROCm 7.2.1, no Docker):

1. `scripts/setup_server.sh` installs Node 22, creates a Python venv, downloads Qwen3-8B via ModelScope, and builds llama.cpp with HIP for `gfx1100`.
2. **Stable profile (verified)**: serve the F16 GGUF with llama-server on 127.0.0.1:8000 — `--ctx-size 8192 --n-gpu-layers 99 --parallel 4`, full GPU offload, thinking disabled for deterministic agent outputs.

3. **Quantized bonus profile**: serve the Q4_K_M GGUF with the same server, enabled only after the frozen quality gate passes.
4. The endpoint host PID is exported as `VLLM_PID` so the bridge can attest the workload is GPU-resident via `amd-smi`.
5. The Pi bridge and the Gradio console connect to the loopback endpoint only.

Engine selection note. vLLM 0.26.0 ships no ROCm wheel and its PyPI wheel is CUDA-only; on this Docker-less gfx1100 instance a vLLM deployment would require a lengthy source build. llama.cpp with HIP is fully supported on RDNA3, builds in minutes, and serves the identical OpenAI-compatible API, so it is the verified engine for this submission. Both paths keep the same agent stack unchanged.

No container runtime is required; both engine paths are source/venv based, which is the realistic deployment route on a Docker-less Radeon instance.

5. Optimization description for inference speed on AMD Radeon GPU

Optimization is treated as an auditable experiment, never a claim:

- **Baseline capture**: fixed prompt set, warm-up excluded, per-request TTFT, total latency, and decode tokens/second recorded to `benchmarks/results/baseline.json`.
- **Levers applied and re-measured**:
  - flash attention and parallel request slots in llama-server;
  - bounded context (`--ctx-size 8192`) and compact JSON specialist outputs;
  - tool-first routing so telemetry math never consumes model tokens;
  - disabled thinking mode so decode tokens go to the answer, not internal chains.
- **Quantized comparison**: Q4_K_M GGUF vs BF16, gated by the frozen quality check.
- **Evidence integrity**: the Performance Lab accepts only JSON records that include raw samples, the prompt hash, VRAM peak, and amd-smi attestation. Numbers that are not measured are marked absent, never estimated.

Before/after numbers from these profiles are committed in `benchmarks/results/` and summarized on the poster and in the demo video.

Measured results on the competition instance (Qwen3-8B, fixed 5-prompt set, streaming, 128 max tokens, warm-up excluded):

Profile	Mean TTFT	Decode tok/s	VRAM peak	Quality gate
Baseline (F16 GGUF, default flags)	56.2 ms	43.6	17.0 GB	n/a
Optimized (F16 GGUF, flash-attn + 8 slots)	57.1 ms	43.6	17.0 GB	n/a
Quantized (Q4_K_M GGUF, flash-attn + 8 slots)	37.5 ms	88.7	6.5 GB	passed

The quantized profile doubles decode throughput (2.03x) and lowers TTFT by 33% while passing the frozen quality gate. The flash-attention / parallel-slot profile keeps single-stream latency neutral while enabling the four specialist agents to be served concurrently without queueing, which is the operating mode of the multi-agent loop (a full five-agent orchestration round was measured end-to-end at ~15 s).